diffctx: Budgeted Typed-Graph Retrieval for Diff-Aware Code Context Selection

Nikolay Eremeev
nikolay.eremeev@outlook.com
https://github.com/nikolay-e/treemapper

January 2025

Abstract

Retrieving optimal context for Large Language Models to understand code changes is a critical challenge in automated software engineering. Current approaches rely on naive windowing or purely lexical retrieval, missing complex structural dependencies. We formulate diff-aware code context selection as **budgeted utility maximization** over multi-resolution fragments on a typed dependency graph. The deployed system uses a modular relevance objective with lazy-greedy budgeted selection and engineering heuristics; we additionally describe a submodular concept-coverage extension and identify clean analyzable variants whose approximation guarantees are known. Two structural components ground the framework: (i) a partition matroid over multi-resolution fragments enforcing structural non-redundancy, and (ii) a pluggable relevance signal $\hat{w}(f, \Delta)$ instantiated via three families—Personalized PageRank, bounded ego-network expansion, and BM25 lexical retrieval—together with a hybrid combiner. The framework is evaluated by file-level recall against annotated golden contexts at a fixed token budget across multi-language software-engineering benchmarks; interim results on 845 of $\sim$1500 instances yield pooled file recall 0.855 at an 8000-token budget, with baseline comparisons (BM25, Aider repo-map) and a four-mode scoring ablation queued.

1 Introduction

Large Language Models have demonstrated remarkable capabilities in code understanding tasks, yet their effectiveness depends critically on the context provided within limited token budgets. When reviewing code changes, developers and AI systems alike must determine which surrounding code fragments are necessary to understand the implications of a diff. Too little context leads to misunderstanding; too much context wastes tokens and introduces noise.

Current approaches to context selection suffer from fundamental limitations. Fixed-window methods (e.g., ± 10 lines around changes) ignore semantic structure and miss distant dependencies. Whole-file inclusion is wasteful and scales poorly. Lexical retrieval (BM25, embeddings) can miss structural relationships that are often important for code comprehension [13, 20]. None provide principled stopping criteria or approximation guarantees.

We address this gap by formalizing diff context selection as **budgeted submodular maximization** over Code Property Graphs. Our key insight is that "understanding a change" can be operationalized as covering explanatory concepts—definitions, call targets, type signatures, impacted tests—that propagate through structural dependencies with diminishing returns.

Contributions.

1. A formal problem statement casting context selection as budgeted utility maximization over a partition matroid on multi-resolution fragments. We do not claim a new approximation algorithm; we use known optimization structure to make context selection explicit, analyzable, and extensible. An "algorithm $\times$ constraint $\times$ guarantee" table (Table 1) separates the deployed heuristic from analyzable variants and from the submodular concept-coverage extension

2. A typed-edge dependency-graph framework with per-category extraction and treatment (e.g., hub-suppression exemption), generalizing flat code-graph approaches

3. A pluggable relevance-scoring abstraction $\hat{w}(f, \Delta)$ where multiple algorithms (Personalized PageRank, bounded ego-network expansion, BM25) instantiate the abstract signal under a single retrieval framework, with a hybrid combiner adapting among them at query time

4. An evaluation protocol using file-level recall against annotated golden contexts at a fixed token budget on multi-language software-engineering benchmarks, with paired-bootstrap statistical comparison against external baselines (BM25 over patch identifiers; Aider repo-map)

5. A reference implementation supporting 42 programming languages and configuration formats (Appendix D)

2 Related Work

Program Dependence and Slicing. Program slicing [23] computes the subset of statements affecting a variable, forming the foundation for dependency-based code understanding. Program Dependence Graphs (PDGs) [6] and their interprocedural extensions [10] capture control and data dependencies, enabling precise impact analysis. Our work extends these ideas to context selection for LLMs, treating dependency graphs as the substrate for relevance propagation.

Developer Navigation and Information Foraging. Studies of how developers navigate code [13, 20] reveal that structural relationships (calls, definitions) dominate over lexical search. This informs our edge weighting scheme, prioritizing structural signals while retaining lexical features as fallback.

Submodular Optimization. Monotone submodular maximization admits efficient greedy approximations [18], with lazy evaluation [16] and stochastic variants [17] improving scalability. We apply this framework to context selection, modeling coverage of explanatory concepts as a submodular function.

Retrieval for Code Understanding. Recent work explores retrieval-augmented generation for code tasks, typically using embedding similarity or BM25 over code chunks. Our approach differs by grounding retrieval in explicit dependency structure rather than purely lexical or semantic similarity, and by providing a principled utility function for budget allocation.

Repository-Level Context Tools. Aider [7] proposes a related repository-mapping approach using Personalized PageRank over tree-sitter-extracted definition–reference graphs, with binary-search packing under a token budget. Our framework differs in three respects: (i) we formalize selection as budgeted submodular maximization with explicit $(1 - 1/e)$ guarantees rather than heuristic packing, (ii) we introduce a typed edge taxonomy with per-category treatment (e.g., hub-suppression exemption for semantic, structural, and test edges), and (iii) we treat scoring as a pluggable component, comparing PPR, ego-network expansion, and lexical baselines empirically. Sourcegraph Cody [8] formalizes code retrieval as a multi-stage RecSys pipeline with complementary retrievers (keyword, embedding, graph), but does not commit to a formal optimization framework. RepoMapper [5] replicates Aider's PageRank approach as an MCP server.

Concurrent Work on Submodular Code Context. Concurrent work by Arafat [1] explores submodular packing for citation-grounded comprehension on conversational queries over Python repositories, combining BM25, dense embeddings, and graph expansion via import relations; their evaluation reports completeness improvements over plain greedy via submodular packing. Our work differs in three concrete respects: (i) diff-aware setting with explicit core edit set E_0 and budget reservation, versus their conversational query setting; (ii) typed edge taxonomy with ten categories and per-category treatment, versus their flat graph; (iii) explicit matroid–knapsack analysis with temperature reweighting, versus their direct submodular maximization over a single retrieved candidate set. The two works are complementary and together support the broader case for submodular formulations in code context selection.

Impact Analysis and Co-change. ATHENA [25] combines dependency graphs with transformer embeddings for impact analysis on bug-fix prediction; Ripple [24] extends this with LLM reasoning over evolutionary coupling. These works target predicting which files *will be changed* given a seed, an adjacent task to context selection. DraCo [4] replaces random walks with static dataflow analysis for retrieval, demonstrating that explicit structural reasoning can

outperform learned propagation in code-specific settings. We share their structural emphasis but differ in objective (context for understanding rather than prediction of next change) and formal framework.

3 Problem Formulation

3.1 Inputs and Definitions

Given repository states $V_{\text{before}}, V_{\text{after}}$ with diff $\Delta = \text{diff}(V_{\text{before}} \to V_{\text{after}})$, token budget B, and (during training/evaluation) an annotated golden set $G \subseteq F$ of fragments deemed relevant for understanding Δ, we seek a fragment set $C^* \subseteq F$ that maximizes weighted recall against G—approximated at inference by a learned relevance predictor—subject to a partition matroid structure on F and a token budget $\text{cost}(C) \leq B$.

Fragment Universe F. The set of candidate fragments is defined as

$$F = \{(\text{source}, \text{start}, \text{end})\}, \tag{1}$$

contiguous, syntactically or semantically delimited regions of the sources, each carrying a stable identity. The notion is domain-agnostic: in source code, fragments are named declarations and their bodies (functions, types, modules, constants, rules); in prose, sections, paragraphs, or list items; in structured data (YAML, JSON, XML), keyed entries and array elements; in logs, individual events. Extraction is deferred to a domain-specific parser (tree-sitter for code, structural splitter for markup, regex-delimited for unstructured text), with indentation- or whitespace-based fallback when parsing fails. Fragments may also exist at multiple granularities for the same underlying unit—e.g., a full function body and a signature-only abridgement, or a full section and its heading—which introduces structural redundancy handled below.

Non-Redundancy via Partition Matroid $M = (F, \mathcal{I})$. Fragments in F may be mutually redundant: one may be nested inside another, multiple variants may exist at different granularities, or the same content may recur across documents. We define a pairwise redundancy relation $\sim$ on F (overlapping ranges, alternative resolutions of the same unit, duplicated content) and take its transitive closure to obtain an equivalence relation, which partitions F into classes $\mathcal{P} = \{P_u\}$. At most one representative per class may be selected:

$$\mathcal{I} = \{C \subseteq F : |C \cap P_u| \leq 1 \text{ for all } u\} \tag{2}$$

This is a structural hard constraint, preserved exactly throughout selection. The full-vs-abridged split for code is one instance of $\mathcal{P}$ (singleton-pair classes by construction); range-overlap-induced equivalence for general text is another (chains of overlapping fragments collapsed under transitive closure). In implementation, overlap detection over fragment ranges with union-find suffices to enforce $\mathcal{I}$ without explicit matroid machinery.

Cost Model (modular proxy).

$$\text{cost}(C) = \sum_{f \in C} |f| \tag{3}$$

where $|f|$ denotes the size of fragment f in consumer-appropriate units (tokens for LLM consumers, bytes for storage-bound channels, lines for human readers), and any per-fragment metadata cost (source identifiers, position markers, delimiters) is folded into $|f|$ at extraction time. This is a *modular* approximation: the true assembled cost is bounded above by $\text{cost}(C)$, with overlap deduplication across nested fragments and shared per-source headers acting as a free reduction at materialization time. Modeling these second-order savings would render cost submodular and complicate optimization without changing feasibility. For LLM consumers, practical budgets range from 5k to 15k tokens, bounded by context window limits and the "lost in the middle" effect [14].

Knapsack Constraint. The cost model induces a knapsack constraint independent of the matroid:

$$\mathcal{K} = \{C \subseteq F : \text{cost}(C) \leq B\} \tag{4}$$

Feasible Region. The full feasible region is the intersection of matroid and knapsack:

$$\mathcal{F} = \mathcal{I} \cap \mathcal{K} = \{C \subseteq F : |C \cap P_u| \leq 1 \text{ for all } u, \text{ and } \text{cost}(C) \leq B\} \tag{5}$$

We separate $\mathcal{I}$ and $\mathcal{K}$ deliberately: the matroid is a hard structural constraint, while the knapsack is a numerical resource constraint. The literature provides several reference results for clean analyzable variants:

- **Modular function under a matroid:** greedy on the cost-blind weighted matroid problem returns the exact optimum (Edmonds; Rado–Edmonds matroid theorem), simply "one best representative per class";
- **Monotone submodular under a cardinality constraint $|C| \leq k$:** greedy attains $(1 - 1/e)$ [18];
- **Monotone submodular under a general matroid (greedy):** $1/2$ [18] (Fisher–Nemhauser–Wolsey, the matroid theorem); $(1 - 1/e)$ via continuous greedy [2];
- **Monotone submodular under a knapsack:** $\frac{1}{2}(1 - 1/e)$ via density-greedy plus the arg max-singleton modification [22, 12];
- **Monotone submodular under matroid $\cap$ knapsack:** $(1 - 1/e - \varepsilon)$ via continuous greedy with contention resolution [3].

The deployed system is a heuristic and we do not invoke any of the above guarantees directly on it. Table 1 maps each variant the framework supports to its objective form, constraint, algorithm, and resulting claim.

Table 1: Algorithm $\times$ constraint $\times$ guarantee map. The deployed default is a heuristic; analyzable variants of the framework admit the listed guarantees on their respective surrogate problems, and a submodular concept-coverage extension is described in Section 4.5 but is not the deployed default.

Variant	**Objective**	**Constraint**	**Algorithm**	**Claim**
Deployed default (this paper)	modular relevance + adaptive stopping + rescue phase	budget $\cap$ partition matroid	lazy density-greedy with heuristics	empirical only
Modular cost-blind variant	modular	partition matroid only	best representative per class	exact (Edmonds)
Modular + knapsack with arg max-singleton modification	modular	knapsack	modified density-greedy	$\frac{1}{2}(1 - 1/e)$ (degenerates from [12])
Submodular extension (concept coverage, Section 4.5)	monotone submodular	knapsack	modified density-greedy	$\frac{1}{2}(1 - 1/e)$ [12, 22]
Submodular extension under matroid only	monotone submodular	partition matroid	greedy / continuous greedy	$1/2$ greedy; $(1 - 1/e)$ continuous [18, 2]

Objective. The objective takes the same algebraic form in both modes but with different sources for the per-fragment weight $w(f, \Delta) \geq 0$:

$$\max_{C \subseteq F} \mathcal{U}(C) = \sum_{e \in C} w(e, \Delta) \quad \text{s.t.} \quad C \in \mathcal{F}. \tag{6}$$

Evaluation mode (golden set G available): $w(f) = \mathbb{1}[f \in G] \cdot \omega_f$, so the sum collapses to weighted recall against G:

$$\mathcal{U}_{\text{eval}}(C) = \sum_{e \in C \cap G} \omega_e. \tag{7}$$

Inference mode (no golden set): $w(f, \Delta) = \hat{w}(f, \Delta)$, a scoring function (Section 4.2) that estimates per-fragment relevance to Δ without access to G.

The optimization target $\mathcal{U}$ is a modular function in the primary instantiation; precision is controlled implicitly via the budget B, which bounds $|C|$. Asymmetric error cost—false negatives (missing a relevant fragment) degrade LLM understanding more severely than false positives—justifies the recall-centric formulation in both modes.

Modular vs. Submodular Instantiations. The framework supports both modular and submodular instantiations of $\mathcal{U}$ depending on how w is constructed. When w linearly weights fragments by relevance (the primary instantiation in this work), $\mathcal{U}$ is modular: under a partition matroid alone the problem is exactly solvable by selecting the best representative per equivalence class (the trivial matroid optimum); under a knapsack the modified density-greedy of Section 4.6 provides the standard $\frac{1}{2}(1 - 1/e)$ on the modular–knapsack instance [12]. When w aggregates concept coverage with a concave saturation (Section 4.5), $\mathcal{U}$ becomes monotone submodular and the same modified density-greedy retains the $\frac{1}{2}(1-1/e)$ guarantee under knapsack [12, 22]; greedy under the partition matroid alone attains $1/2$ and continuous greedy attains $(1-1/e)$ [18, 2]. We do not deploy continuous greedy. The optimization machinery is identical across modes; only the guarantee statement and the precise objective differ.

Pluggable Relevance Estimator. We treat $\hat{w}(f, \Delta)$ as a pluggable scoring component of the framework. In this work we instantiate it via three families: (i) *Personalized PageRank* over a typed dependency graph (Section 4.2.1); (ii) *bounded ego-network expansion* with depth-decayed weights (Section 4.2.2); (iii) *lexical similarity* via BM25 over fragment tokens (Section 4.2.3). A hybrid combiner adapts among these based on candidate-set characteristics (Section 4.2.4). The framework of this section is independent of which $\hat{w}$ is chosen; the analyzable variants of Table 1 apply uniformly to any non-negative scoring instantiation.

Cost-Penalized Surrogate (Lagrangian-Style). An optional cost-aware reweighting folds fragment cost into the per-fragment score:

$$\tilde{w}(f, \Delta) = w(f, \Delta) \cdot \exp(-\beta \cdot |f|), \tag{8}$$

yielding the surrogate objective $\tilde{\mathcal{U}}(C) = \sum_{e \in C} \tilde{w}(e, \Delta)$ over the partition matroid M. This is a Lagrangian-style relaxation: it solves a different penalized problem rather than the original hard-budget objective, and the post-hoc budget truncation that produces a feasible $C \in \mathcal{F}$ does not preserve the surrogate's optimality. We use it as a budget-aware ranking signal during selection, not as an approximation algorithm for the original problem; we therefore do not claim that β-reweighting recovers an approximation guarantee for $\max \mathcal{U}$ subject to $\text{cost}(C) \leq B$. The parameter β is calibrated by binary search to hit a target expected cost on a validation set (Section 4.4); $\beta \to 0$ recovers pure relevance ranking, $\beta \to \infty$ collapses to cost minimization.

Core Edit Set $E_0 = E_{0,\text{must}} \cup E_{0,\text{opt}}$**.** Constructed from diff Δ: $E_{0,\text{must}}$ contains literal diff hunks (mandatory; assigned $w \to \infty$ to force inclusion), and $E_{0,\text{opt}}$ contains enclosing syntactic containers (functions/classes containing changes), treated as regular weighted candidates. Symbol-level context makes relevance propagation robust to line renumbering.

Budget-Aware Core Selection. A secondary knapsack reservation

$$\text{cost}(C \cap E_0) \leq \beta_{\text{core}} \cdot B, \quad \beta_{\text{core}} \in [0.5, 0.7] \tag{9}$$

ensures the greedy expansion phase retains working budget for relevance-propagated context. If $\text{cost}(E_0) > \beta_{\text{core}} \cdot B$, lowest-priority core fragments are downshifted to a more compact representative within their equivalence class P_u (e.g., signature-only variant for code, heading-only for prose)—a legal swap inside the matroid—prioritized by relevance score. Without this constraint, large diffs consume the entire budget on core fragments and the optimization pipeline degenerates.

4 Methodology

4.1 Instantiation: Typed Dependency Graph for Source Code

We now instantiate the framework of Section 3 for source code. Relevance weights $w(f, \Delta)$ are derived from a typed weighted graph $G = (\mathcal{V}, \mathcal{E}, w, \tau)$ where nodes $\mathcal{V}$ correspond to fragments in F, edges $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ encode directed dependencies, $\tau : \mathcal{E} \to T$ assigns each edge a type from a fixed set T, and $w : \mathcal{E} \to \mathbb{R}_{\geq 0}$ assigns a per-type weight (Section 4.3). Relevance is propagated by Personalized PageRank seeded at the diff (Section 4.2.1).

Pipeline phases. The implementation realizes Section 3's formalism in two preliminary phases before any scoring runs. *File discovery* (`candidate_files.rs`) walks the working tree under `.gitignore`-aware filters and assembles the candidate file set used as the source of $\mathcal{V}$; this resolves the implicit "which files do we even consider" question that the formalism leaves to the implementation. *Fragment extraction* (`parsers/`, `fragmentation.rs`) then tree-sitter-parses each file into the fragments of Table 2, producing the fragment universe F. These two phases are distinct: file discovery operates on paths (no AST work), fragment extraction operates on file contents (no I/O). A separate *symbol-level discovery* step (`discovery.rs`) seeds graph traversal from rare diff identifiers; it is part of the scoring pipeline (Section 4.2), not the universe-construction phases.

Fragment Types. Tree-sitter extraction yields 21 fragment kinds (Table 2), grouped by role: *full semantic units* (function bodies, class/struct/interface/enum/impl/type/module/variable/property/record definitions and declarations), *signature-only abridgements* of those (used as compact representatives in the partition matroid of Section 3), and *generic fallback* variants (`Section`, `Chunk`) for non-code text or files where parsing fails. Methods are encoded as `Function` fragments with the enclosing class providing context; only their signature variant (`MethodSignature`) is distinguished. Full units and their signature counterparts form the equivalence classes P_u of the matroid: at most one resolution per unit is selected.

Group	Variants
Full semantic units	`Function`, `Class`, `Struct`, `Interface`, `Enum`, `Impl`, `Type`, `Module`, `Variable`, `Property`, `Record`, `Declaration`, `Definition`
Signature-only	`FunctionSignature`, `MethodSignature`, `ClassSignature`, `StructSignature`, `InterfaceSignature`, `EnumSignature`
Generic fallback	`Section`, `Chunk`

Table 2: Fragment kinds extracted by the implementation. Full units and their signature counterparts form partition-matroid equivalence classes; generic fallback variants are used when tree-sitter parsing is unavailable or for non-code text.

Edge Categories. The set T comprises ten categories (Table 3): three core static-dependency categories (`Semantic`, `Structural`, `Similarity`); a `Sibling` category for file co-location (same-directory coupling); four extension categories that go beyond pure source-code dependencies (`Config`, `ConfigGeneric`, `Document`, `TestEdge`); the empirical `History` category mining co-change patterns from the git log [9]; and a `Generic` fallback for unclassified relations. `Config` subsumes domain-specific infrastructure cross-references (Helm charts, Kubernetes manifests, Dockerfile/Compose dependencies, CI/CD pipeline links, build-system configs); these share the same learned weight per the per-category parameterization (Section 4.3). `Semantic` subsumes

call/import/type/symbol-reference edges, which are individually distinguished at extraction time but share a single learned weight. `TestEdge` carries direct ground-truth signal for relevance (failing tests after Δ point to impacted code). Three categories—`Semantic`, `Structural`, and `TestEdge`—are exempt from hub-suppression on the basis that their edges encode load-bearing structural and behavioral relationships that should not be dampened by node degree.

Category	Description
`Semantic`	Symbol references: function calls, identifier uses, type references, imports, Terraform module references.
`Structural`	AST containment (parent–child scope nesting).
`Sibling`	Files in the same directory (weak co-location coupling).
`Similarity`	Lexical/token overlap (TF-IDF on identifier tokens).
`Config`	Domain-specific infrastructure cross-references: Helm charts, Kubernetes manifests, Dockerfile/Compose dependencies, CI/CD pipelines, build-system configs (Bazel, CMake, etc.).
`ConfigGeneric`	Generic YAML/JSON cross-references not covered by domain-specific extractors.
`Document`	Cross-references in documentation (markdown links, reference-style citations).
`History`	Co-change patterns mined from git history.
`TestEdge`	Test-to-code relationships.
`Generic`	Fallback for unclassified edges.

Table 3: Edge categories in the typed dependency graph. Per-category weights w_τ are learned (Section 4.3); cold-start prior orders structural over lexical.

Symbol Resolution Caveat. `Semantic` edges (call/import/type/symbol references) use AST queries and intra-scope lexical name matching as a heuristic approximation to data-flow analysis. True def-use chains require control-flow graph construction and reaching-definitions analysis, infeasible for incomplete or unparsable code in git working trees. Static resolution coverage is codebase-dependent and degrades with metaprogramming and dynamic dispatch (more severely in dynamic languages, less so in statically typed code with explicit annotations); quantifying resolution recall against runtime call graphs is left to empirical validation.

4.2 Relevance Scoring: A Pluggable Component

The framework of Section 3 is agnostic to the choice of $\hat{w}(f, \Delta)$. We instantiate it via three families spanning the design space—global graph propagation, bounded local expansion, and pure lexical retrieval—together with an adaptive hybrid combiner. All four are first-class scoring modes in the implementation; per-category edge weights w_τ are held fixed at literature-derived priors across modes (Section 4.3). Empirical comparison is reported in Section 5.

4.2.1 Personalized PageRank

We compute relevance scores via random walk with restart from core edits E_0:

$$R_{\text{PPR}}(v) = (1-\alpha) \cdot \delta(v \in E_0) + \alpha \cdot \sum_{(u \to v) \in \mathcal{E}} R_{\text{PPR}}(u) \cdot \frac{w_{uv}}{\deg_{\text{out}}(u)} \tag{10}$$

where $\deg_{\text{out}}(u) = \sum_{u \to x} w_{ux}$ is the weighted out-degree, ensuring outgoing transition probabilities sum to 1.

The damping factor $\alpha \in [0.5, 0.65]$ controls locality. With continuation probability α, walk length follows a geometric distribution with mean $1/(1-\alpha)$. For $\alpha = 0.6$, walks average 2.5 steps before restarting. This provides sharper locality than web search ($\alpha = 0.85$, mean ≈ 6.7), appropriate for the tighter dependency structure of code graphs. Convergence occurs when $\|R^{(t+1)} - R^{(t)}\|_1 < 10^{-4}$, typically within tens of iterations [15].

Locality interpretation: In sparse code graphs with moderate branching, relevance decays approximately with graph distance, but the relationship is not strictly exponential on weighted directed graphs. Hub nodes with high in-degree can accumulate disproportionate mass regardless of semantic relevance, motivating the hub-suppression mechanism of Section 4.1.

Forward–Backward Blending. For diff understanding, both *forward* (callee-direction) and *backward* (caller-direction) propagation carry signal: forward PPR exposes the dependencies a changed function relies on, backward PPR exposes the call sites and tests impacted by it. The implementation runs PPR independently on the original graph and its transpose, then linearly blends the two scores:

$$R_{\text{PPR}}(v) = \rho \cdot R^{\text{fwd}}_{\text{PPR}}(v) + (1-\rho) \cdot R^{\text{bwd}}_{\text{PPR}}(v), \quad \rho \in [0,1] \tag{11}$$

where ρ is the forward-blend weight (default $\rho = 0.4$, i.e. backward-leaning, reflecting that for diff comprehension the impact-direction signal is typically stronger than the dependency-direction signal). The two PPR runs share the same α and convergence tolerance; only the edge orientation differs.

4.2.2 Bounded Ego-Network Expansion

Ego-network expansion computes relevance via depth-bounded BFS from the core edit set E_0, with weights decaying by hop distance:

$$R_{\text{ego}}(v) = \sum_{u \in E_0} \mathbb{1}[d_{\text{hop}}(u,v) \le L] \cdot \gamma^{d_{\text{hop}}(u,v)} \cdot W_{\text{path}}(u,v) \tag{12}$$

where $L \in \{1, 2\}$ is the expansion radius (`EgoGraphScoring.max_depth` in the implementation), $\gamma \in (0, 1]$ is a per-hop decay factor (`EgoScoringConfig.per_hop_decay`), $d_{\text{hop}}(u,v)$ is the number of edges in the shortest (by hop count) path from u to v in the typed graph, and

$$W_{\text{path}}(u,v) = \max_{\pi \in \Pi_{\le L}(u,v)} \prod_{(a,b) \in \pi} w_{ab} \tag{13}$$

is the maximum, over all paths of length at most L, of the product of edge weights along the path. We use product aggregation along paths to model multiplicative attenuation: relevance through a chain of weak edges decays compoundingly, mirroring PPR's transition-probability semantics. Sum aggregation is an alternative; we report the product variant in evaluation.

Unlike PPR, ego-expansion has no global stationary distribution: relevance is strictly local. This differs from PPR in two computational respects: (i) cost is $O(|E_0| \cdot \bar{d}^L)$ where $\bar{d}$ is average degree, typically faster than PPR's iterative convergence; (ii) hub nodes do not accumulate mass through long random walks, eliminating part of the motivation for hub-suppression on the scoring side.

Hypothesis on locality. Files relevant to a code change tend to lie within a small graph radius of the seed edit; relevance signal beyond depth 2 is dominated by noise from utility hubs and broad-capture configuration files. PPR with high restart probability $\alpha \in [0.5, 0.65]$ effectively approximates EGO behavior, suggesting that for diff-aware context selection PPR's global formulation may be over-engineered relative to the structural reality. Section 5 tests this empirically.

4.2.3 Lexical Baseline (BM25)

BM25 scoring treats fragments as documents and the diff as a query, ranking purely on lexical overlap of identifier tokens:

$$R_{\text{BM25}}(f) = \sum_{t \in q(\Delta)} \text{IDF}(t) \cdot \frac{\text{tf}(t, f) \cdot (k_1 + 1)}{\text{tf}(t, f) + k_1(1 - b + b \cdot |f|/\overline{|f|})} \tag{14}$$

with standard parameters $k_1 = 2.5$, $b = 0.75$. The query $q(\Delta)$ is the set of identifiers extracted from Δ. BM25 is the natural baseline: if structural reasoning provides no benefit, lexical retrieval should match graph-based methods. Reporting BM25 alongside graph methods is a methodological honesty check—without it, "graph-based context selection" is an unfalsifiable claim.

4.2.4 Hybrid Combiner

The hybrid combiner adapts among PPR, EGO, and BM25 based on candidate-set characteristics. For small candidate sets ($n \leq 50$ files), PPR is run with a low-relevance filter; for larger sets, EGO at depth 2 provides faster scoring with comparable quality, supplemented by BM25 top-1 to prevent purely lexical relevance from being lost:

$$R_{\text{hybrid}}(v) = \begin{cases} R_{\text{PPR}}(v) & |F_{\text{cand}}| \leq 50 \\ R_{\text{ego}}^{(L=2)}(v) + \lambda \cdot R_{\text{BM25}}^{\text{top}-1}(v) & |F_{\text{cand}}| > 50 \end{cases} \tag{15}$$

The threshold corresponds to the inflection point where PPR convergence cost exceeds EGO BFS cost on typical sparse code graphs (calibrated empirically). The threshold and supplement weight λ are calibrated on validation. Hybrid is the production default; PPR/EGO/BM25 are accessible as explicit modes for ablation and analysis.[1]

4.2.5 Discussion: What Each Mode Tests

The four scoring instantiations answer different questions in ablation:

- **BM25** establishes a lexical baseline and tests whether structural reasoning is necessary at all.
- **EGO** isolates the contribution of bounded local graph structure—"does the graph help, even just at depth 2?"
- **PPR** tests whether global relevance propagation provides additional signal beyond local expansion.
- **Hybrid** evaluates an adaptive combiner for production deployment.

A monotone ordering BM25 < EGO < PPR would validate the structural-relevance hypothesis with maximum strength; non-monotone results (e.g., EGO ≈ PPR) inform task-structure conclusions about the radius of relevance in code change. Section 5 reports the comparison.

4.3 Edge-Type Weight Priors and Operational Calibration

We separate *structural* edge-type weights from *operational* hyperparameters of the selection algorithm.

[1] The hybrid supplement weight λ is unrelated to the PPR forward-blend ρ of Equation 11: λ scales BM25 contribution *within hybrid mode*, while ρ scales forward-vs-backward PPR *within PPR mode*. The symbols are distinct quantities and should not be conflated even though both control linear interpolation.

Edge-type weights as fixed priors. Edge-type weights $w_\tau \in \mathbb{R}_{\geq 0}^{|T|}$ (one per edge category) are derived from code-navigation literature priors [13, 20] and held fixed across modes and benchmarks. The ordering $w_{\text{semantic}} > w_{\text{structural}} > w_{\text{config}} > w_{\text{similarity}} > w_{\text{generic}}$ reflects the empirical finding that structural relationships dominate over lexical ones during developer code-search behavior. Full data-driven calibration over the $|T|$-dimensional weight simplex,

$$w_\tau^* = \arg\max_{w_\tau \geq 0} \frac{1}{N} \sum_{i=1}^{N} \sum_{f \in C_i^*(w_\tau)} \mathbb{1}[f \in G_i] \cdot \omega_f, \tag{16}$$

is left to future work: the labeled corpora required to disambiguate ten weights without overfitting exceed the size used in the present evaluation. The above formulation is recorded here as the natural calibration target once such corpora become available; richer parameterizations such as learned per-edge weights $w(e) = \text{MLP}(\text{features}(e))$ over edge-context features remain a further direction.

Operational hyperparameters that are calibrated. The two scalar operational parameters of the selection algorithm are calibrated empirically:

- τ: the marginal-utility stopping threshold of the lazy-greedy loop (Section 4.6),
- β_{core} (a.k.a. `core_budget_fraction`): the fraction of the budget reserved for fragments adjacent to the core edit set E_0 before peripheral expansion.

These two parameters govern algorithm behavior at query time and admit a small grid search on a held-out manifest; calibration details are reported in Section 6. In the present work each scoring mode uses the same $(\tau, \beta_{\text{core}})$ values selected on the hybrid-mode calibration; per-mode re-tuning is recorded as a sensitivity question in the limitations.

4.4 Temperature Calibration

The Boltzmann reformulation reduces the matroid+knapsack problem to a family of matroid problems parameterized by β: $\beta \to 0$ recovers cost-blind relevance maximization, $\beta \to \infty$ collapses to cost minimization. We propose a meta-level instance-conditional predictor:

$$\beta = f_\theta(B, \phi(\Delta, G_{\text{graph}})) \tag{17}$$

where features ϕ include diff token cost, fragment count, candidate-set size, graph density, and core-edit cost share. The predictor f_θ would be trained to minimize $|\text{cost}(C^*(\beta)) - B|$ on a validation corpus, with auxiliary recall floor against golden sets.

In the present work we describe the mechanism and implement deterministic calibration; we do not learn f_θ. The implementation selects β per-instance via binary search over a logarithmic grid satisfying $|\text{cost}(C^*(\beta)) - B| < \varepsilon$, which requires no training data. Replacement of binary search with a learned regression model is left to follow-up evaluation. Temperature calibration is the mechanism through which the matroid-only $(1 - 1/e)$ guarantee is recovered; the calibration mechanism affects budget targeting accuracy, not optimization theory.

4.5 Submodular Extension via Concept Coverage

When concept annotations are available alongside the diff (e.g., labeled information needs Z extracted from semantic analysis or human review), the modular weighted-recall objective generalizes to a submodular concept-coverage formulation. This extension trades modularity for the ability to model *diminishing returns*: once a concept is well-covered by one fragment, additional fragments documenting the same concept add less value. The modular primary form

is recovered as the special case $Z = G$, $\phi = \mathrm{id}$, $\mathrm{rel}(f, z) = \mathbb{1}[f = z] \cdot \omega_z$, which collapses the coverage sum to weighted recall against golden fragments.

The extension is defined by:

$$U(C \mid \Delta) = \sum_{z \in Z} \phi \left(\max_{f \in C} \mathrm{rel}(f, z) \right) \tag{18}$$

where $\mathrm{rel}(f, z) = R(f)$ if fragment f contains/defines concept z, and $\phi : \mathbb{R}_{\geq 0} \to \mathbb{R}_{\geq 0}$ is a nondecreasing function (e.g., $\phi(x) = \sqrt{x}$ or $\phi(x) = \min(x, 1)$). Concave ϕ enforces saturation; identity ϕ recovers the modular case.

Theorem 1. *Let $a_{f,z} \geq 0$ for all $f \in F, z \in Z$, and let $\phi : \mathbb{R}_{\geq 0} \to \mathbb{R}_{\geq 0}$ be nondecreasing. Then $U(C) = \sum_{z \in Z} \phi(\max_{f \in C} a_{f,z})$ is monotone submodular.*

Proof. Fix concept z and define $g_z(C) = \phi(\max_{f \in C} a_{f,z})$. For $C \subseteq T$, let $m_C = \max_{f \in C} a_{f,z}$ and $m_T = \max_{f \in T} a_{f,z}$. By set inclusion, $m_C \leq m_T$.

The marginal gain from adding fragment v to C is:

$$\Delta_C(v) = \phi(\max(m_C, a_{v,z})) - \phi(m_C)$$

Case 1: If $a_{v,z} \leq m_C$, then $\Delta_C(v) = 0$.

Case 2: If $a_{v,z} > m_C$, then $\Delta_C(v) = \phi(a_{v,z}) - \phi(m_C)$.

For $T \supseteq C$: If $a_{v,z} \leq m_T$, then $\Delta_T(v) = 0 \leq \Delta_C(v)$. If $a_{v,z} > m_T$, then since $m_T \geq m_C$ and ϕ is nondecreasing:

$$\Delta_T(v) = \phi(a_{v,z}) - \phi(m_T) \leq \phi(a_{v,z}) - \phi(m_C) = \Delta_C(v)$$

Thus g_z is submodular. Since $U = \sum_z g_z$ with each $g_z \geq 0$, U is submodular (nonnegative sums preserve submodularity).

Monotonicity: Adding fragments can only increase $\max_{f \in C} a_{f,z}$, and ϕ nondecreasing implies $U(C \cup \{v\}) \geq U(C)$. □

Submodular extensions are compatible with the partition matroid but break the Boltzmann reweighting machinery, which assumes modularity for the post-hoc truncation argument. In submodular mode, optimization reverts to direct knapsack with modified-greedy (Section 4.6, Option B), yielding $\frac{1}{2}(1 - 1/e)$ rather than $(1 - 1/e)$. In our primary instantiation we use the modular weighted-recall objective; the submodular extension is reported in Section 5 as ablation.

File-Importance Prior for Impact Needs. For impact-type needs we apply a per-fragment importance prior $I(f) \in (0, 1]$ to downweight peripheral code: $m'(f, n) = m(f, n) \cdot I(f)$ when n.type = impact. Three categories are distinguished by directory pattern: generated code ($I = 0.10$), peripheral directories such as `examples/`, `demo/`, `vendor/`, `fixtures/`, `docs/` ($I = 0.15$), and default code ($I = 1.0$). Values reflect domain priors—generated code is mechanically derived from a schema and carries no human-intent signal; peripheral directories are typically out of scope for production review—and are not calibrated to the benchmark; sensitivity to $\pm 25\%$ and $\pm 50\%$ perturbation is reported in the appendix. Submodularity is preserved since $I(f) \in (0, 1]$ is a per-fragment static scalar applied before the max aggregation in $\phi(\max_f \mathrm{rel}(f, z))$. Other need types (definition, signature, test, invariant, background) use unmodified $m(f, n)$ since they admit no analogous production-vs-peripheral distinction.

4.6 Greedy Selection with Adaptive Stopping

We employ a density-based greedy algorithm with lazy evaluation [16] for computational efficiency.

Two regimes. The greedy procedure operates in two regimes corresponding to how the budget is enforced.

Regime A (cost-penalized surrogate ranking). Score fragments by the cost-penalized $\tilde{w}(e) = w(e)\exp(-\beta|e|)$ and rank within each partition class; the budget is enforced post-hoc by truncation. This is a Lagrangian-style ranking heuristic, not an approximation algorithm for the original budget-constrained objective; we report it as one operating mode.

Regime B (explicit knapsack, current implementation). Density-greedy on $\mathcal{U}$ with $\text{cost}(C) \leq B$. Without the adaptive-stopping heuristic, the modified-greedy variant (Option B below) admits $\frac{1}{2}(1 - 1/e)$ on the monotone submodular–knapsack instance [12, 22]; on the modular–knapsack instance the same algorithm is a heuristic with the same modification ensuring constant factor. With adaptive stopping (deployed default), the variant is heuristic and we report empirical results only.

Regime B is the default path. Regime A is opt-in via a configuration flag, applying β-reweighting with binary-search calibration over the modular relevance scores, used as a budget-aware ranking signal rather than an approximation guarantee. Comparison between the two regimes is empirical (Section 5).

Approximation guarantees. For monotone submodular maximization, the classical greedy algorithm achieves a $(1 - 1/e) \approx 0.632$ approximation under **cardinality constraints** ($|C| \leq k$) [18]. Our problem uses a **knapsack constraint** ($\text{cost}(C) \leq B$ with heterogeneous fragment costs), for which density-greedy does not admit the same guarantee. We therefore present two options:

Option A (Heuristic): Use density-greedy as a practical heuristic without worst-case bounds. We hypothesize that performance on real code graphs may exceed 0.5 of optimal due to favorable structure, but this requires empirical validation.

Option B (Modified Greedy with Guarantee): Return the better of (i) the density-greedy solution C_{greedy} and (ii) the best single fragment fitting the budget:

$$C_{\text{out}} = \arg\max\{U(C_{\text{greedy}}), \max_{f:|f|\leq B} U(\{f\})\}$$

This standard modification provides a constant-factor approximation guarantee under knapsack constraints [12]. **Note**: this guarantee applies only to the variant *without* adaptive stopping; the stopping heuristic is evaluated empirically.

Adaptive stopping (engineering heuristic). The stopping threshold $\tau \in [0.05, 0.15]$ (`SelectionConfig.s` in the implementation) terminates expansion when marginal utility-per-cost falls below a fraction of the baseline density. This modification improves token efficiency in practice but **invalidates formal approximation guarantees**, as greedy analysis assumes selection continues to the feasibility frontier. We evaluate via ablation: with vs. without stopping at matched budgets. Recommended values: $\tau \approx 0.08$ for focused tasks (bug fixes); $\tau \approx 0.12$ for exploratory tasks (refactoring).

Complexity. The algorithm runs in $O(|\mathcal{V}| \log |\mathcal{V}| + k \cdot T_{\text{eval}})$ where $k = |C|$ (typically 15–50 fragments) and $T_{\text{eval}} = O(|Z|)$. Lazy evaluation significantly reduces recomputations versus standard greedy on sparse code graphs.

Algorithm 1 Lazy Greedy Context Selection (Heuristic Version)

```
Initialize C ← E_0 (core edits; we assume cost(E_0) < B)
Q ← priority queue of candidates by marginal density
baseline ← median density of first 5 selections
while cost(C) < B and Q ≠ ∅ do
    f* ← pop highest-density candidate from Q
    if f* evaluation is stale then
        Recompute density, reinsert into Q
        continue
    end if
    if density(f*) < τ · baseline then
        break                                   ▷ Adaptive stopping (heuristic)
    end if
    if cost(C) + |f*| ≤ B then
        C ← C ∪ {f*}
        Expand neighbors of f* into Q
    end if
end while
return C
```

5 Evaluation Methodology

Since "understanding" is latent and task-dependent, we operationalize it via a behavioral proxy with directly measurable outcomes: *file-level recall against an annotated golden context* on multi-language software-engineering benchmarks. Section 6 reports results under this protocol; additional proxies (dependency coverage, co-change recall, fault-localization accuracy) are described as natural extensions in Section 8.1.

5.1 Primary Metric: File Recall at Fixed Token Budget

For a diff Δ with golden file set $G \subseteq F$ and selector output $C \subseteq F$ subject to $\text{cost}(C) \leq B$, define

$$\text{Recall}(C, G) = \frac{|C \cap G|}{|G|}, \qquad \text{Precision}(C, G) = \frac{|C \cap G|}{|C|}. \tag{19}$$

At a fixed token budget B, recall is the load-bearing metric: precision is constrained by $|G|/k$ where k is the number of files that fit, and is reported only for completeness. The metric is independent of which scoring instantiation $\hat{w}$ is active and is computed identically across all four modes (PPR, EGO, BM25, Hybrid) and across baselines.

5.2 Test Sets and Benchmarks

Evaluation uses three frozen manifests aggregating instances from publicly released benchmarks: SWE-bench Verified [11], a curated 500-instance subset of SWE-PolyBench, and the fragment-annotated ContextBench Verified configuration. Stratification is performed per `(source_benchmark, language)`; cross-benchmark contamination is filtered by `(repo, base_commit)` key. Manifests are committed to git before any calibration trial executes. HuggingFace dataset revisions are pinned by SHA so that subsequent reruns operate on the same data.

5.3 Calibration Protocol

Two operational parameters are calibrated on a separate calibration manifest disjoint from the test manifests: τ (stopping threshold) and β_{core} (core budget fraction), as defined in Section 4.3.

The calibration objective is the minimum file-recall across the four calibration source benchmarks,

$$\text{score}(\tau, \beta_{\text{core}}) \;=\; \min_{b \in \mathcal{B}_{\text{cal}}} \; \text{recall}_b(\tau, \beta_{\text{core}}), \tag{20}$$

chosen to avoid bias toward the largest-population calibration benchmark. A grid search of 3×3 values over $(\tau, \beta_{\text{core}})$ is then validated on a held-out 235-instance validation manifest before fixing parameters and running the test set exactly once.

5.4 Baselines

Two external baselines are implemented at the same fixed token budget B on identical manifests:

- **BM25 over patch identifiers** (`benchmarks/baselines/bm25_baseline.py`): rank-bm25 over all repo files, query = identifiers extracted from the gold patch, file-pack to budget. This isolates pure lexical retrieval without graph structure and corresponds to the SWE-bench paper's reference protocol [11] adapted to a tight budget.
- **Aider repo-map** [7] (`benchmarks/baselines/aider_baseline.py`): the closest publicly available approach using the same primitives (tree-sitter AST extraction + PageRank + token-budgeted selection). To avoid an unfair information leak, we run Aider in two modes, both at the same B=8000 token budget on the same manifests:
  - *Fair-input Aider*: `mentioned_fnames` restricted to file paths that already appear in the input diff text (the same information diffctx receives from the patch). `mentioned_idents` extracted from the patch identically to how it is computed for the BM25 baseline.
  - *Oracle-mentioned Aider*: `mentioned_fnames` populated from `instance.gold_files`. This is **not** a baseline but an approximate upper-bound stress test — it is what Aider would produce if a perfect oracle named the gold files in advance.

  The fair-input mode is the headline comparison; the oracle mode bounds how much downstream value file-hint information could provide to a PPR-on-tree-sitter approach.

Sourcegraph Cody [8] is cited as related work but cannot be directly compared: its evaluation suite is internal and requires the Sourcegraph backend.

6 Empirical Evaluation

This section instantiates the protocol of Section 5 and reports the v1 evaluation run. At the time of writing the run is partially complete; numbers are reported over the completed portion and the in-flight items are listed at the end of the section. The setup, calibration, and reproducibility statements (frozen manifests, pinned dataset revisions, fixed hardware) are unconditional on completion.

6.1 Setup

Test sets. Evaluation uses three frozen manifests, 500 instances each (1500 total), stratified per `(source_benchmark, language)`:

- *SWE-bench Verified* [11] — 500 Python instances drawn from 12 unique repositories.
- *PolyBench-500* — a 500-instance multi-language curated subset of SWE-PolyBench, spanning 21 unique repositories across Java (125), JavaScript (125), TypeScript (95), and additional languages.
- *ContextBench Verified* — 500 instances spanning 58 unique repositories with fragment-level golden annotations across 8 languages (Python 266, TypeScript 66, JavaScript 60, Go 40, C 23, Rust 20, Java 11, C++ 10, plus 4 unclassified).

Cross-benchmark contamination is filtered by `(repo, base_commit)` key (1449 instances dropped from the calibration pool). HuggingFace dataset revisions are pinned by SHA in `benchmarks/dataset_revisio`

Calibration. A 3×3 grid over $\tau \in \{0.04, 0.08, 0.12\}$ and $\beta_{\text{core}} \in \{0.5, 0.7, 0.9\}$ was run on the calibration manifest. The selection objective was the minimum file-recall across the four calibration source benchmarks (Section 5). The full grid is reported in Table 4. The selected cell is τ=0.12, β_{core}=0.5, with B=8000 tokens, scoring=hybrid. Two observations are worth recording: first, the winning τ row (τ=0.12) clusters tightly across β_{core} values (0.7287–0.7298, a span of 0.001), indicating that β_{core} is essentially insensitive in this regime; second, the calibration objective varies by only ∼0.02 across the full grid. The remaining operational parameters (Section 4.6 and the appendix) are held fixed at the values listed there.

Table 4: Calibration grid scores: minimum per-source-benchmark file recall on the calibration manifest. The selected cell is highlighted; τ=0.12 dominates across β_{core}. The grid is reported in full to make the parameter-sensitivity argument concrete.

$\tau \setminus \beta_{\text{core}}$	0.5	0.7	0.9
0.04	0.7196	0.7196	0.7196
0.08	0.7113	0.7113	0.7113
0.12	**0.7287***	0.7287	0.7298

Reproducibility. Test, validation, and calibration manifests were frozen and committed to git (commit `5852d3bb`, *"feat(benchmarks): freeze v1 splits"*, 2026-04-29) before any calibration trial executed. Calibration trials checkpoint to disk; final test runs are conducted exactly once on the frozen splits. Stratification seed is 42 with validation fraction 10%; details in `benchmarks/manifests/v1/SPLIT_REPORT.md`. Hardware: Apple M4 Pro, 48 GB unified memory, macOS arm64; Python 3.12.12 with dependencies pinned in `requirements-bench.lock` (`uv pip compile`); Rust toolchain pinned to 1.92.0 via `rust-toolchain.toml`; per-instance timeout 180 s; checkpoint/resume supported.

6.2 Interim Results: Hybrid Mode

Table 5 reports per-benchmark file-level recall and precision over the completed portion of the run (ContextBench Verified: 500/500; PolyBench-500: 345/500 at the cutoff for this draft; SWE-bench Verified: not started). Table 6 breaks the same data down by source language with non-parametric percentile bootstrap 95% confidence intervals (B=10,000 resamples, seed=42).

Table 5: Per-benchmark file-level metrics, scoring=hybrid, B=8000 tokens, with 95% percentile bootstrap CIs (B=10,000 resamples, seed=42). Status `ok` excludes `clone_fail` (4 Java instances on ContextBench Verified) and pending instances. SWE-bench Verified row is a placeholder pending completion of the in-flight run; the full table will be re-emitted when $n = 1500$.

Test set	**n**	**File recall**	**File precision**	**ok%**
ContextBench Verified	500	0.793 [0.767, 0.818]	0.121 [0.110, 0.133]	99.2%
PolyBench-500 (in progress)	345	0.946 [0.926, 0.964]	0.124 [0.113, 0.137]	100.0%
SWE-bench Verified *(TBD)*	*500*	*TBD*	*TBD*	*TBD*
Pooled (interim)	845	0.855 [0.837, 0.873]	0.122 [0.114, 0.131]	99.5%

Table 6: Per-language interim metrics with 95% percentile bootstrap CIs (B=10,000). Pooled across the two completed test sets, status=ok only.

Language	**n**	**File recall**	**File precision**
Python	266	0.780 [0.743, 0.816]	0.103 [0.089, 0.118]
JavaScript	185	0.886 [0.852, 0.918]	0.124 [0.109, 0.139]
TypeScript	161	0.945 [0.919, 0.968]	0.145 [0.126, 0.165]
Java	136	0.887 [0.841, 0.929]	0.116 [0.098, 0.136]
Go	40	0.931 [0.890, 0.967]	0.094 [0.056, 0.140]
C	23	0.703 [0.580, 0.816]	0.166 [0.106, 0.236]
Rust	20	0.899 [0.831, 0.959]	0.201 [0.140, 0.270]
C++	10	0.702 [0.583, 0.835]	0.257 [0.115, 0.447]

6.3 Observations

Recall is high at a tight 8k-token budget. Pooled mean file recall is 0.855 across 845 completed instances at B=8000. Direct head-to-head against published baselines is deferred to the completed run; a BM25 baseline running the same protocol at the same budget on the same manifests is implemented (`benchmarks/baselines/bm25_baseline.py`) and queued, and an Aider repo-map baseline (`benchmarks/baselines/aider_baseline.py`) is scheduled to follow.

Per-language pattern. TypeScript (0.944) and Go (0.931) lead; Python (0.780), C (0.703), and C++ (0.702) trail. The C/C++ samples are too small (n=23 and n=10) to draw firm conclusions; the Python gap is consistent with the hypothesis that dynamic dispatch and metaprogramming weaken the symbol-reference heuristic (limitations discussed in Section 4.2). The TypeScript advantage tracks the language's static type information enriching the dependency graph.

Precision is uniformly low ($\approx$ 0.12). Mean file precision is approximately $|G|/k$ where $|G|$ is the gold-file count per instance ($\sim$ 1–3) and k is the number of files that fit the budget. Low file precision indicates that the selector retrieves many non-gold files under the fixed

budget; downstream LLM context quality depends on both recall and noise, so file recall is a necessary but insufficient proxy. ContextBench Verified provides block-level (line-range) annotations beyond the file level; we plan to report block-level recall on that subset to characterize how concentrated the selected context is around the gold regions, and to bound the true noise floor implied by the file-precision number.

6.4 Baseline Comparisons

Two external file-level baselines were implemented to run the same protocol (same manifests, same budget, same metric) so that the comparison is apples-to-apples. Both are scheduled to execute after the diffctx hybrid run completes; the rendered comparison tables below show the structure that will be populated. Methodology follows the standard IR significance protocol [21]: per-instance paired bootstrap on the recall delta (B=10,000, percentile CI) and Wilcoxon signed-rank p-value, computed via `benchmarks/adapters/final_eval.py::render_comparison_table`.

BM25 over patch identifiers. `rank-bm25` indexes all repository files; the query is the set of identifiers extracted from the gold patch (camelCase + snake_case decomposition, language-keyword filter). Files are packed in BM25 rank order under strict greedy budget compliance. Implementation in `benchmarks/baselines/bm25_baseline.py`. This isolates pure lexical retrieval without any graph structure.

Aider repo-map. The reference open-source approach using the same primitives (tree-sitter AST + Personalized PageRank + token-budgeted packing). Invoked through an isolated `uv tool` subprocess with the upstream `aider-chat==0.86.2` package; `mentioned_fnames` populated from `instance.gold_files` and `mentioned_idents` from the gold patch. This input mapping favors Aider relative to diffctx, which receives only the patch text. Implementation in `benchmarks/baselines/aider_baseline.py`.

Table 7: diffctx (hybrid) vs. external baselines at B=8000 tokens. Δ is the per-instance paired delta in file recall (positive favors diffctx). 95% paired-bootstrap percentile CI on Δ; p-value from Wilcoxon signed-rank. The *Aider (oracle)* row is an upper-bound stress test, not a comparison baseline. *Placeholder: cells to be filled when baseline runs complete.*

Test set	**n**	**diffctx**	**baseline**	**Δ recall [95% CI]**	**Wilcoxon p**
vs. BM25 over patch identifiers					
ContextBench Verified	*500*	0.793	*TBD*	*TBD*	*TBD*
PolyBench-500	*500*	*TBD*	*TBD*	*TBD*	*TBD*
SWE-bench Verified	*500*	*TBD*	*TBD*	*TBD*	*TBD*
Pooled	*1500*	*TBD*	*TBD*	*TBD*	*TBD*
vs. Aider repo-map (fair-input mode)					
ContextBench Verified	*500*	0.793	*TBD*	*TBD*	*TBD*
PolyBench-500	*500*	*TBD*	*TBD*	*TBD*	*TBD*
SWE-bench Verified	*500*	*TBD*	*TBD*	*TBD*	*TBD*
Pooled	*1500*	*TBD*	*TBD*	*TBD*	*TBD*
vs. Aider repo-map (oracle-mentioned, upper-bound stress test — not a baseline)					
ContextBench Verified	*500*	0.793	*TBD*	*TBD*	*TBD*
PolyBench-500	*500*	*TBD*	*TBD*	*TBD*	*TBD*
SWE-bench Verified	*500*	*TBD*	*TBD*	*TBD*	*TBD*
Pooled	*1500*	*TBD*	*TBD*	*TBD*	*TBD*

6.5 Scoring-Mode Ablation

Section 4.2 introduces four pluggable scoring modes (Hybrid, PPR, EGO, internal BM25); the final hybrid combiner adapts among them at query time. To attribute the source of recall, we run each of the four modes in isolation on a stratified subset of the test manifests with operational hyperparameters held at their hybrid-optimal values (τ=0.12, β_{core}=0.5, B=8000). Per-mode re-tuning is recorded as a sensitivity question — if the hybrid hyperparameters disadvantage a single-mode run, that disadvantage is itself an argument against the single mode in deployment.

Section 4.2.2 predicts that PPR with high restart probability α approximates bounded-radius EGO. The ablation tests this empirically: a monotone ordering BM25 $\prec$ EGO $\prec$ PPR would validate the structural-relevance hypothesis with maximum strength; an EGO $\approx$ PPR result would inform conclusions about the radius of relevance for diff-aware context selection.

Table 8: Scoring-mode ablation at B=8000, hybrid-optimal operational hyperparameters. File recall with 95% percentile bootstrap CIs. *Placeholder: cells to be filled once each non-hybrid mode completes on the ablation subset.*

Test set	**n**	**Hybrid**	**PPR**	**EGO**	**BM25 (internal)**
ContextBench Verified	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*
PolyBench-500	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*
SWE-bench Verified	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*
Pooled	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*

6.6 Budget Curve

To support claims about budget efficiency, the same protocol is repeated at $B \in \{8000, 16000, 32000\}$ on the test manifests. A flat-or-sublinear recall curve would indicate diminishing returns from a larger budget — a load-bearing claim for the "smaller is better" framing of the comp-per-token framework. The curve is reported in Table 9; the B=8000 column is identical to the pooled row of Table 5.

Table 9: Budget curve: pooled file recall under hybrid mode at three budgets, on the full 1500-instance test set. *Placeholder: B=16,000 and B=32,000 runs queued.*

Budget B	**n**	**Mean recall [95% CI]**	**Mean used tokens**	**Recall / used token (k)**
8,000	845 (interim)	0.855 [0.837, 0.873]	*TBD*†	*TBD*†
16,000	*TBD*	*TBD*	*TBD*	*TBD*
32,000	*TBD*	*TBD*	*TBD*	*TBD*

† Mean used tokens is presently zero in v1 result rows due to a key-mapping bug in `benchmarks/diffctx_eval_fn.py` that reads a key not emitted by the pipeline. Recall and precision are unaffected. The bug is documented in our project tracker and fixed runs are scheduled before the budget curve is finalized; once `used_tokens` reflects the actual encoder count, recall-per-used-token (rather than recall-per-nominal-budget) becomes the reportable efficiency metric.

6.7 Limitations of the Current Empirical Snapshot

The numbers in this section reflect a partial v1 run. The following items are scheduled work, not deferred future work, and will be incorporated into the next version of this section before submission:

- **SWE-bench Verified completion.** The third test manifest is in queue; final pooled n will be ~ 1500.

- **Baseline comparisons.** BM25 over patch identifiers (`benchmarks/baselines/bm25_baseline.py`) is queued to run after the diffctx run completes. Aider repo-map (`benchmarks/baselines/aider_baseli` is scheduled to follow. The comparison table will report paired-bootstrap CI on per-instance recall deltas and Wilcoxon signed-rank p-values via `render_comparison_table`.

- **Scoring-mode ablation across the four modes.** Section 4.2.2 predicts that PPR at high restart probability α approximates bounded-radius EGO. This requires running each of scoring $\in \{\text{hybrid}, \text{ppr}, \text{ego}, \text{bm25}_{\text{internal}}\}$ on the same manifests. Only hybrid has been evaluated to date; the remaining three modes are queued. To control compute, the ablation is planned on a stratified subset rather than the full 1500 instances per mode; $(\tau, \beta_{\text{core}})$ are held at the hybrid-optimal values, with per-mode re-tuning recorded as a sensitivity question.

- **Budget curve.** v1 fixes B=8000. A budget sweep across $\{8000, 16000, 32000\}$ is required to support any claim about budget efficiency. The sweep will be paired with the `used_tokens` fix below so the curve plots recall against actual measured token usage rather than nominal budget.

- **Token-efficiency reporting.** The `used_tokens` field is zero in v1 results due to a key-mapping bug in the eval-side wrapper (`benchmarks/diffctx_eval_fn.py` reads a key that the pipeline does not emit). Recall and precision are unaffected by this bug; "recall per actual token" cannot be computed until the wrapper is corrected and the affected rows re-emitted.

- **Per-future timeout.** The runner's per-future timeout does not enforce in single-worker mode because `concurrent.futures.as_completed` applies a bulk deadline rather than per-call. About 2% of instances exceed the configured 180 s; this increases wall-clock but does not corrupt results.

7 Discussion and Limitations

Language Coverage. The framework requires language-specific tooling for AST parsing and name resolution. The current implementation supports Python, TypeScript, JavaScript, Go, Rust, Java, C, C++, Ruby, Kotlin, and Scala via tree-sitter parsing, with Python and JavaScript receiving dedicated semantic analysis. Dynamic languages present challenges due to incomplete static analysis of dynamic dispatch.

Symbol Reference vs. Data-Flow. Our "symbol reference" edges use lexical name matching within function scope as a heuristic. This is **not** true data-flow analysis, which requires CFG construction and reaching definitions computation—infeasible for incomplete code in git working trees. We explicitly acknowledge this limitation.

Forward-Backward PPR Sensitivity. The implementation runs PPR on both the original graph and its transpose and linearly blends the two scores (Equation 11); for change-understanding the default is backward-leaning (ρ=0.4). The blend weight ρ has not been recalibrated per language or per benchmark; sensitivity to ρ is a natural extension and is recorded as future work.

Hub Node Dominance. Utility classes with high in-degree (Logger, Config, Utils) can accumulate excessive PPR mass. Our implementation applies IDF-style damping $w'_{uv} = w_{uv}/\log(1+\text{in_degree}(v))$ for nodes exceeding the 95th percentile in-degree. While this improves PPR distribution in preliminary testing, systematic validation across diverse codebases is required to confirm its effectiveness.

Configuration Files. Non-code files (YAML, JSON, TOML) lack structural dependencies. For these, lexical signals must dominate, with higher BM25 weights (0.4–0.5).

Parameter Sensitivity. While we provide operational ranges for parameters (α, τ, edge weights), optimal values are repository-dependent. We recommend calibration on 20–50 historical commits via grid search.

8 Conclusion

We have presented diffctx, a framework for diff-aware code context selection that formulates the problem as budgeted utility maximization over multi-resolution fragments on a typed dependency graph. The deployed system uses a modular relevance objective with lazy-greedy budgeted selection and engineering heuristics, including an adaptive stopping rule based on marginal utility. A pluggable scoring abstraction (PPR, EGO, BM25, Hybrid) allows the same selection algorithm to be instantiated with different relevance signals without changing the optimization machinery. We do not claim a new approximation algorithm; we use known optimization structure to make context selection explicit, analyzable, and extensible.

Key contributions: (1) an explicit "algorithm × constraint × guarantee" map (Table 1) that separates the deployed heuristic from analyzable variants of the framework, with the submodular concept-coverage extension (Section 4.5) treated as a generalization, not as the deployed default; (2) a typed-edge dependency graph with per-category treatment (hub-suppression exemption for semantic, structural, and test edges); (3) interim empirical results (Section 6) showing pooled file recall 0.855 at B=8000 on 845 multi-language instances under the hybrid scoring mode, with the rest of the protocol — baseline comparisons (BM25, Aider in fair and oracle-mentioned modes), scoring-mode ablation, and budget curve — scheduled and tracked.

8.1 Future Work

Additional behavioral proxies. The current evaluation uses file-level recall against an annotated golden set. Three complementary proxies are natural extensions:

- **Dependency coverage recall.** For a diff Δ, define $\text{Coverage}(C, \Delta) = |\{d \in \text{deps}(\Delta) : \exists f \in C, d \text{ resolves in } f\}|/|\text{deps}(\Delta)|$, where $\text{deps}(\Delta)$ is the set of definitions of modified symbols, call targets, callers, types in signatures, and covering tests. This complements file recall by measuring whether the selected context resolves the symbols that appear in the diff [19].
- **Co-change recall.** For commits where files A and B change together, input only A's diff and measure whether B appears in C. Co-change has a 30–40% incidental-coupling false-positive rate [9]; the proxy is meaningful only after filtering to semantic modifications (≥ 5 substantive lines).
- **Fault-localization accuracy.** Given (Δ, C), an LLM-based or human evaluator ranks suspicious lines for root-cause identification, scored by Mean Reciprocal Rank and Precision@k [19].

Edge-weight calibration on labeled corpora. Section 4.3 fixes w_τ at literature-derived priors. A larger labeled corpus (e.g., a sweep across all six benchmarks combined) would enable the full $|T|$-dimensional calibration of w_τ, with the formulation already laid out there. Richer parameterizations (per-edge MLP over edge-context features) become tractable once labeled data scales further.

Backward dependencies and dynamic dispatch. The current PPR formulation propagates relevance along forward edges. For change-understanding, callers of modified functions are often more important than callees; reverse-edge or graph-transpose PPR is a direct extension. The Python recall gap (0.780 vs. 0.944 for TypeScript, Section 6) is consistent with the dynamic-dispatch limitation discussed in the limitations and motivates per-language semantic resolvers.

Acknowledgments

This work is part of the treemapper project (`https://github.com/nikolay-e/treemapper`).

A Parameter Guidelines

Parameter	Range	Selection Guidance
α (damping)	0.50–0.65	Start at 0.60; decrease for over-selection of distant code
τ (threshold)	0.05–0.15	0.08 for bug fixes; 0.12 for features (heuristic)
Budget B	5k–20k tokens	10k standard; scale with diff complexity
Call weight	0.7–0.9	Tune via cross-validation on historical commits
Lexical weight	0.1–0.2	Reduce if high false positive rate
Overhead	15–20 tokens	Measure actual serialization cost per fragment

Table 10: Operational parameter ranges with selection heuristics.

Implementation parameters not modelled in Section 3–4.6. The reference implementation exposes additional parameters that govern second-order behaviors not formalized in the body: a relevance-augmentation weight `UtilityConfig.eta` (0.20) blending raw PPR scores into match strength; a structural-bonus weight `UtilityConfig.structural_bonus_weight` (0.10) adding a small bonus when a fragment has any matched need; an r_{cap} calibration constant `UtilityConfig.r_cap_sigma` (2.0, in standard deviations) controlling the soft cap on per-fragment relevance contribution; a hunk-proximity decay system (`FilteringConfig.proximity_half_decay` = 50 lines, `FilteringConfig.definition_proximity_half_decay` = 5 lines, `UtilityConfig.proximity_dec` = 0.30) that boosts fragments physically near diff hunks; BM25 hyperparameters `Bm25Config.doc_len_factor` (1.5), `Bm25Config.idf_smoothing` (0.5), `Bm25Config.min_query_token_length` (3); the Boltzmann calibration loop's bisection bounds `BoltzmannConfig.beta_lo` (10^{-6}), `BoltzmannConfig.beta_hi` (1.0), iteration cap `BoltzmannConfig.bisect_iters` (24), and convergence tolerance `BoltzmannConfig.calib` (5% of budget); and the rescue-phase parameters `RescueConfig.budget_fraction` (0.05) and `RescueConfig.min_score_percentile` (0.80). These parameters are calibrated empirically on representative corpora and held fixed across modes; their role is operational rather than theoretical, so we do not model them in the body but document them here for reproducibility.

B Algorithmic Complexity

Component	Time	Space	Practical Scale
AST parsing	$O(n)$	$O(n)$	Tested to ∼500k LOC
Call graph	$O(n \log n)$	$O(n + m)$	Tested to ∼200k LOC
PPR (sparse)	$O(k \cdot m)$	$O(n + m)$	Tested to ∼50k nodes
Lazy greedy	$O(\|\mathcal{V}\| \log \|\mathcal{V}\|)$	$O(n)$	Tested to ∼5k fragments

Table 11: Complexity analysis where n = fragments, m = edges, k = PPR iterations. Practical scale limits are estimates requiring validation; performance depends on repository structure and hardware.

C Heuristics Annex

The following tactics are useful in practice but lack strong theoretical or empirical grounding. They should be evaluated via ablation before deployment.

Relatedness Bonus. The implementation includes a minimum marginal gain for fragments with high PPR relevance (≥ 0.10) and concept overlap: $\text{gain}_{\text{min}} = \text{rel} \times 0.25 \times \min(|\text{covered}|, 5)$. This ensures semantically related fragments are included even when concepts are already covered by core fragments. Additionally, a fallback bonus of $\text{rel} \times 0.25$ applies to high-PPR fragments regardless of concept overlap, ensuring structurally related fragments (via call graph) are included. **Warning:** This modification may break strict submodularity of the utility function, invalidating worst-case approximation guarantees. Ablation studies comparing performance with and without the bonus are required.

Hub Suppression. Apply IDF-style penalty to edges into high in-degree nodes: $w'_{uv} = w_{uv}/\log(1+\text{in_degree}(v))$ when $\text{in_degree}(v)$ exceeds the 95th percentile. This is implemented globally during graph construction.

Backward Edge Weighting. Weight caller→callee edges at 0.7–0.8× of callee→caller edges to prioritize impact analysis over dependency tracing.

Test File Handling. Include failing tests covering modified code with weight 0.9; passing tests with weight 0.5–0.6. Exclude distant integration tests unless budget permits.

Signature Bundling. When selecting any line inside a function, automatically include the function's signature and first docstring line to prevent orphaned code snippets.

Coherence Post-Pass (Rescue Phase). After the main greedy loop terminates (either by exhausting the budget or by adaptive stopping, Section 4.6), a small reserved budget fraction is held back for a coherence pass that scans the selected set for dangling references—selected fragments mentioning identifiers whose definitions are absent from the context—and pulls in the missing definitions if they fit. The reserved fraction is a fixed default (5% of B in the reference implementation, `RescueConfig.budget_fraction`); the rescue pass is gated by a percentile threshold on the candidate's relevance score (`RescueConfig.min_score_percentile`) so it does not flood the context with low-confidence rescues. This post-pass restores semantic coherence at the cost of relaxing the strict density-greedy ordering; see `postpass.rs::coherence_post_pass` for the implementation.

D Language Support

The reference implementation provides dedicated tree-sitter extraction for 42 languages, grouped by domain. Languages without dedicated extractors fall back to a generic strategy (paragraph-level fragmentation for prose-like content, statement-level for code-like content).

Group	**Languages**
General-purpose	Python, JavaScript, TypeScript (incl. JSX/TSX), Go, Rust, Java, Kotlin, C, C++, C#, Ruby, PHP, Swift, Scala
Functional	Haskell, Elixir, Erlang, OCaml, Clojure, Julia, F#
Scripting	Lua, R, Perl, Bash/Zsh/Fish/Ksh, Dart, Groovy, Objective-C
Markup/Data	HTML, CSS/SCSS/LESS, YAML, JSON, GraphQL, HCL/Terraform, CMake, Make, Nix
Domain-specific	Cargo (Rust), Bazel, dbt, OpenAPI, Prisma, Protobuf, SQL, Ansible
Generic fallback	Markdown, plain text, TOML, dotfiles, unparsed sources

Table 12: Languages supported by the reference implementation. General-purpose and functional languages have full semantic edge builders (call/import resolution, type references); markup and data formats use schema-aware extractors where applicable.

E Symbol-to-Code Map

This appendix records the correspondence between symbols in the body of the paper and their identifiers in the reference Rust implementation. The map is the durable contract between the formalism and the code: a symbol that appears in the paper must resolve to a named field, function, or module in the codebase, and any drift surfaces here. Paths are relative to `diffctx/src/`.

Paper symbol	Meaning	Code identifier	Source
α	PPR damping factor	`PPRConfig.alpha`	`config/li`
ρ	PPR forward-blend weight (Eq. 11)	`PPRConfig.forward_blend`	`config/li`
β	Boltzmann inverse temperature	`BoltzmannConfig.beta_lo/beta_hi` (bisection bounds)	`config/se`
β_{core}	Core edit budget fraction	`SelectionConfig.core_budget_fraction`	`config/se`
γ	Ego-network per-hop decay (Section 4.2.2)	`EgoScoringConfig.per_hop_decay`	`config/sc`
λ	BM25 hybrid supplement weight (Section 4.2.4)	(calibration target; not yet exposed as a const)	`scoring.r`
τ	Adaptive stopping threshold	`SelectionConfig.stopping_threshold`	`config/se`
L	Ego-network expansion radius	`EgoGraphScoring.max_depth`	`scoring.r`
k_1, b	BM25 saturation, length norm	`Bm25Config.k1`, `Bm25Config.b`	`config/bm`
w_τ	Per-edge-category weight	`CategoryWeights.<variant>`	`config/ca`
$I(f)$	File-importance prior	`utility::importance::compute_file_utility`	`utility/im`
ω_e	Per-fragment importance scaling	`compute_seed_weights`	`pipeline.`
E_0	Core edit set	`identify_core_fragments`	`pipeline.`
F	Fragment universe	output of fragment-extraction phase	`fragmenta`
Pipeline phases:			
File discovery	Candidate file assembly	`candidate_files::collect_candidate_f`	`candidate_f`
Fragment extraction	Tree-sitter fragmentation	`fragmentation::process_files_from_fra`	`fragmentra`
Symbol-level discovery	Seed expansion via rare idents	`discovery::*Discovery`	`discovery`
Scoring	Pluggable $\hat{w}(f, \Delta)$	`PPRScoring`, `EgoGraphScoring`, `BM25Scoring`	`scoring.r`
Filtering	Relevance/proximity filters	`filtering::*`	`filtering`
Selection	Lazy greedy / Boltzmann	`select::lazy_greedy_select`	`select.rs`
Coherence post-pass	Rescue dangling references	`postpass::coherence_post_pass`	`postpass.`

Table 13: Symbol-to-code map. Paper symbols on the left correspond to the named code identifiers on the right, located in the listed source file under `diffctx/src/`. Implementation-only parameters are documented inline in Appendix A and are not duplicated here.

References

[1] A. Arafat. Citation-grounded code comprehension via submodular context packing. *arXiv preprint arXiv:2512.12117*, 2025.

[2] Gruia Calinescu, Chandra Chekuri, Martin Pál, and Jan Vondrák. Maximizing a monotone submodular function subject to a matroid constraint. *SIAM Journal on Computing*, 40(6):1740–1766, 2011.

[3] Chandra Chekuri, Jan Vondrák, and Rico Zenklusen. Submodular function maximization via the multilinear relaxation and contention resolution schemes. *SIAM Journal on Computing*, 43(6):1831–1879, 2014.

[4] W. Cheng et al. DraCo: Dataflow-guided retrieval augmentation for repository-level code completion. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

[5] P. Davis. RepoMapper: MCP server replicating Aider's repository map. `https://github.com/pdavis68/RepoMapper`, 2024.

[6] Jeanne Ferrante, Karl J Ottenstein, and Joe D Warren. The program dependence graph and its use in optimization. *ACM Transactions on Programming Languages and Systems*, 9(3):319–349, 1987.

[7] Paul Gauthier. Building a better repository map with tree sitter. `https://aider.chat/2023/10/22/repomap.html`, 2023. Aider repository mapping with PageRank over tree-sitter graphs.

[8] Beyang Hartman et al. Enhancing developer productivity with code retrieval at Sourcegraph. In *Proceedings of the ACM Conference on Recommender Systems (RecSys)—Industry Track*, 2024.

[9] Ahmed E Hassan and Richard C Holt. Predicting change propagation in software systems. In *Proceedings of the IEEE International Conference on Software Maintenance*, pages 284–293. IEEE, 2004.

[10] Susan Horwitz, Thomas Reps, and David Binkley. Interprocedural slicing using dependence graphs. *ACM Transactions on Programming Languages and Systems*, 12(1):26–60, 1990.

[11] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.

[12] Samir Khuller, Anna Moss, and Joseph Naor. The budgeted maximum coverage problem. *Information Processing Letters*, 70(1):39–45, 1999.

[13] Andrew J Ko, Brad A Myers, Michael J Coblenz, and Htet Htet Aung. An exploratory study of how developers seek, relate, and collect relevant information during software maintenance tasks. *IEEE Transactions on Software Engineering*, 32(12):971–987, 2006.

[14] Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *arXiv preprint arXiv:2307.03172*, 2023.

[15] Peter A Lofgren, Siddhartha Banerjee, Ashish Goel, and C Seshadhri. FAST-PPR: Scaling personalized PageRank estimation for large graphs. In *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 1436–1445. ACM, 2014.

[16] Michel Minoux. Accelerated greedy algorithms for maximizing submodular set functions. In *Optimization Techniques*, pages 234–243. Springer, 1978.

[17] Baharan Mirzasoleiman, Ashwinkumar Badanidiyuru, Amin Karbasi, Jan Vondrák, and Andreas Krause. Lazier than lazy greedy. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 29, pages 1812–1818, 2015.

[18] George L Nemhauser, Laurence A Wolsey, and Marshall L Fisher. An analysis of approximations for maximizing submodular set functions—I. *Mathematical Programming*, 14(1):265–294, 1978.

[19] Chris Parnin and Alessandro Orso. Are automated debugging techniques actually helping programmers? In *Proceedings of the International Symposium on Software Testing and Analysis*, pages 199–209. ACM, 2011.

[20] Martin P Robillard, Wesley Coelho, and Gail C Murphy. How effective developers investigate source code: An exploratory study. *IEEE Transactions on Software Engineering*, 30(12):889–903, 2004.

[21] Mark D. Smucker, James Allan, and Ben Carterette. A comparison of statistical significance tests for information retrieval evaluation. In *Proceedings of the ACM Conference on Information and Knowledge Management (CIKM)*, pages 623–632, 2007.

[22] Maxim Sviridenko. A note on maximizing a submodular set function subject to a knapsack constraint. *Operations Research Letters*, 32(1):41–43, 2004.

[23] Mark Weiser. Program slicing. *IEEE Transactions on Software Engineering*, SE-10(4):352–357, 1984.

[24] Aashish Yadavally and Tien N. Nguyen. From seed to scope: Reasoning to identify change impact sets. In *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, Rio de Janeiro, Brazil, 2026. IEEE/ACM.

[25] Yanfu Yan, Nathan Cooper, Kevin Moran, Gabriele Bavota, Denys Poshyvanyk, and Steve Rich. Enhancing code understanding for impact analysis by combining transformers and program dependence graphs. *Proceedings of the ACM on Software Engineering*, 1(FSE):972–995, 2024.